

Assignment No. 3

- a. What is histogram and information can be interpreted from histograms?

A **histogram** is a graphical representation of the distribution of numerical data. It consists of **bars**, where each bar represents the **frequency** (count) of data points within a specific range (bin). Unlike bar charts, histograms are used for continuous data.

Information Interpreted from Histograms

1. Data Distribution

- **Normal Distribution:** A symmetric, bell-shaped histogram suggests a normal distribution.
- **Skewed Distribution:**
 - **Right-skewed (positive skew):** More values on the left, with a long right tail.
 - **Left-skewed (negative skew):** More values on the right, with a long left tail.

2. Central Tendency

- The **peak (mode)** of the histogram shows where most data points are concentrated.
- Helps estimate **mean and median** by observing the shape.

3. Spread (Variability)

- A **wide histogram** indicates a large spread (high variability).
- A **narrow histogram** suggests low variability.

4. Outliers and Gaps

- **Outliers** appear as isolated bars far from the main distribution.
- **Gaps** indicate missing data ranges.

5. Symmetry & Modality

- **Unimodal:** One peak (single central value).
- **Bimodal/Multimodal:** Multiple peaks, indicating mixed populations.

Example Use Cases

- **Exam Scores:** Understanding student performance distribution.
- **Stock Market Data:** Identifying volatility in stock prices.
- **Website Traffic:** Analysing visit frequency over time.

b. What information is extracted from boxplots?

A **boxplot (box-and-whisker plot)** is a graphical representation of a dataset's distribution, summarizing key statistical measures. It helps in visualizing spread, central tendency, and outliers efficiently.

Information Extracted from Boxplots

1. Five-Number Summary

- **Minimum:** The smallest data point (excluding outliers).
- **First Quartile (Q1 - 25th percentile):** Median of the lower half of the data.
- **Median (Q2 - 50th percentile):** The middle value of the dataset.
- **Third Quartile (Q3 - 75th percentile):** Median of the upper half of the data.
- **Maximum:** The largest data point (excluding outliers).

2. Interquartile Range (IQR) & Spread

- **IQR = Q3 - Q1**, representing the middle 50% of data.
- A **wide IQR** indicates high variability, while a **narrow IQR** suggests a more clustered dataset.

3. Outliers Detection

- Points **outside the whiskers** (beyond $1.5 * \text{IQR}$) are potential outliers.

- Outliers can indicate **errors, anomalies, or important rare events** in data.

4. Symmetry & Skewness

- **Symmetric Boxplot:** The median is centered within the box, and whiskers are roughly equal.
- **Right-Skewed (Positive Skew):** Median is closer to Q1, with a longer right whisker.
- **Left-Skewed (Negative Skew):** Median is closer to Q3, with a longer left whisker.

5. Comparison of Multiple Distributions

- **Side-by-side boxplots** help compare distributions across categories (e.g., salaries by department).
- Differences in medians, spread, and outliers can be observed easily.

Use Cases

- **Exam Scores:** Comparing student performance across subjects.
- **Stock Market:** Analysing price volatility over time.
- **Medical Studies:** Identifying anomalies in patient health data.

c. What information can be obtained from scatterplot?

A **scatterplot** is a graphical representation of the relationship between two numerical variables. Each point on the plot represents an observation, with one variable on the **x-axis** and the other on the **y-axis**.

Information Obtained from a Scatterplot

1. Relationship Between Two Variables

- **Positive Correlation:** As one variable increases, the other also increases (upward trend).
- **Negative Correlation:** As one variable increases, the other decreases (downward trend).

- **No Correlation:** Points are scattered randomly, showing no clear pattern.

2. Strength of Relationship

- **Strong Relationship:** Points are closely clustered around an invisible trend line.
- **Weak Relationship:** Points are widely spread with no clear pattern.

3. Outliers Detection

- Points that are **far from the overall pattern** may indicate anomalies or errors.

4. Clustering or Grouping

- Scatterplots can reveal **clusters**, indicating subgroups in the data.
- Example: Customers with similar purchasing behaviour forming clusters.

5. Linearity vs. Non-Linearity

- **Linear Relationship:** Points form a straight-line pattern.
- **Non-Linear Relationship:** Curved or complex patterns suggest non-linear dependency.

6. Heteroscedasticity vs. Homoscedasticity

- **Homoscedasticity:** Consistent spread of points across the plot.
- **Heteroscedasticity:** Varying spread (e.g., increasing variance), which can indicate problems in regression analysis.

Use Cases

- **Economics:** Relationship between income and spending.
- **Health:** Correlation between BMI and blood pressure.
- **Marketing:** Effect of ad spending on sales.

d. Explain different techniques for missing values handling

Techniques for Handling Missing Values in Data

Missing values in a dataset can affect analysis and model performance. Here are different techniques to handle them:

1. Deletion Methods

a) Listwise Deletion (Complete Case Analysis)

- Removes entire rows with missing values.
- Used when missing data is **small** and **random**.
- **Downside:** Reduces data size, may lead to bias.

b) Pairwise Deletion

- Uses all available data without removing entire rows.
 - Helpful in **correlation and regression** analysis.
 - **Downside:** Can lead to inconsistent sample sizes.
-

2. Imputation Techniques

a) Mean/Median/Mode Imputation

- **Numerical Data:** Replace missing values with the **mean or median**.
- **Categorical Data:** Replace with the **mode (most frequent value)**.
- **Best for** small missing data percentage.
- **Downside:** Can distort variance and relationships.

b) Forward Fill (ffill) & Backward Fill (bfill)

- **ffill:** Fills missing value with the previous observation.
- **bfill:** Fills missing value with the next observation.
- **Best for** time-series data.
- **Downside:** Can lead to incorrect trends if data is not sequential.

c) Interpolation

- Estimates missing values using **linear, polynomial, or spline interpolation**.
- **Best for** time-series or structured numerical data.
- **Downside:** Assumes a consistent pattern.

d) K-Nearest Neighbors (KNN) Imputation

- Fills missing values based on similar data points (**KNN algorithm**).
- **Best for** datasets with similar patterns.
- **Downside:** Computationally expensive for large datasets.

e) Regression Imputation

- Predicts missing values using **regression models** (e.g., Linear Regression).
- **Best for** when data follows a clear trend.
- **Downside:** Can introduce bias if assumptions are incorrect.

3. Advanced Techniques

a) Multiple Imputation (MICE - Multiple Imputation by Chained Equations)

- Generates multiple estimates for missing values using different models.
- **Best for** complex datasets.
- **Downside:** Computationally intensive.

b) Deep Learning-Based Imputation

- Uses models like **Autoencoders** to reconstruct missing values.
- **Best for** high-dimensional data.
- **Downside:** Requires large amounts of data.

4. Using an Indicator Variable

- Creates a new column marking missing values (0 = present, 1 = missing).
- Allows models to learn patterns of missing data.

- **Best for** cases where missingness is **not random**.
-

Best Approach?

- **Small missing data (<5%) → Mean/Median/Mode Imputation**
- **Time-series data → Forward/Backward Fill or Interpolation**
- **Non-random missing data → Regression or KNN Imputation**
- **Large missing data (>30%) → Consider Dropping the Feature or Using Advanced Methods like MICE**

e. Explain different techniques for categorical feature encoding.

Techniques for Categorical Feature Encoding

Categorical data must be converted into numerical values before being used in machine learning models. Below are various encoding techniques based on data types and complexity.

1. Basic Encoding Techniques

a) One-Hot Encoding (OHE)

- Creates **binary columns** for each category.
- **Example:**

Color: [Red, Blue, Green]

One-Hot Encoding:

Red → [1, 0, 0]

Blue → [0, 1, 0]

Green → [0, 0, 1]

- **Pros:** Works well for nominal data (no order).
- **Cons:** Increases dimensionality for high-cardinality features.

b) Label Encoding

- Assigns a **unique integer** to each category.
- **Example:**

Color: [Red, Blue, Green]

Label Encoding:

Red → 0

Blue → 1

Green → 2

- **Pros:** Simple, memory-efficient.
- **Cons:** May introduce **false ordinal relationships**.

2. Advanced Encoding Techniques

c) Ordinal Encoding

- Assigns integer values **based on order/rank**.
- **Example** (Education Levels):

High School → 1

Bachelor → 2

Master → 3

PhD → 4

Best for: Ordinal categorical data (where order matters).

d) Target Encoding (Mean Encoding)

- Replaces categories with the **mean of the target variable**.
- **Example** (Predicting Loan Default):

City | Default Rate

Mumbai | 0.10

Delhi | 0.15

Chennai | 0.20

- **Best for:** High-cardinality categorical variables.
- **Cons:** Can lead to **data leakage** if not applied correctly.

e) Frequency Encoding

- Replaces categories with their **occurrence frequency**.
- **Example:**

Category | Count

A | 100

B | 200

C | 50

Pros: Reduces dimensionality.

Cons: Doesn't capture relationships between categories.

f) Binary Encoding

- Converts categories to **binary representations**.
- **Example** (for 5 categories):

A → 000

B → 001

C → 010

D → 011

E → 100

Pros: Reduces dimensionality compared to One-Hot Encoding.

Cons: Can be **hard to interpret**.

g) Hash Encoding (Feature Hashing)

- Assigns categories to a **fixed number of hash buckets**.
- **Useful for:** Very high-cardinality categorical features.
- **Downside:** May lead to **hash collisions** (same hash for different categories).

Refer the Loan dataset in the given file and perform the following queries.

1. Identify the most frequent values for all categorical features

```
most_frequent=df.select_dtypes(include=['object','category']).mode().iloc[0]  
print(most_frequent)
```

```
Loan_ID      LP001002  
Gender       Male  
Married      Yes  
Dependents   0  
Education    Graduate  
Self_Employed No  
Property_Area Semiurban  
Loan_Status  Y  
Name: 0, dtype: object
```

2. Give descriptive statistics of numerical features in the dataset. Comment about the distribution of data from it.

```
numerical_data=df[['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Loan_Amount_Term', 'Credit_History']]  
descriptive_stats=numerical_data.describe()  
print(descriptive_stats)
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term \
count	614.000000	614.000000	592.000000	600.000000
mean	5403.459283	1621.245798	146.412162	342.000000
std	6109.041673	2926.248369	85.587325	65.12041
min	150.000000	0.000000	9.000000	12.000000
25%	2877.500000	0.000000	100.000000	360.000000
50%	3812.500000	1188.500000	128.000000	360.000000
75%	5795.000000	2297.250000	168.000000	360.000000
max	81000.000000	41667.000000	700.000000	480.000000

```
Credit_History  
count    564.000000
```

```

mean    0.842199
std     0.364878
min     0.000000
25%     1.000000
50%     1.000000
75%     1.000000
max     1.000000

```

3. Replace the missing values in categorical features using appropriate techniques.

```

categorical_features = df.select_dtypes(include=['object', 'category']).columns
for col in categorical_features:
    df[col].fillna(df[col].mode()[0], inplace=True)
print(df.to_string())

```

```

Loan_ID  Gender  Married  Dependents  Education  Self_Employed  Property_Area
a Loan_Status  Gender_Encoded  Married_Encoded  Dependents_Encoded  Educa
tion_Encoded  Self_Employed_Encoded  Property_Area_Rural  Property_Area_Se
miurban  Property_Area_Urban  Loan_Status_Encoded
0  L1  Male    No      0  Graduate    No      Urban      Y      1
0      0      0      0      0      False      False      True
1
1  L2  Female  Yes      1  Not Graduate    Yes  Semiurban    N
0      1      1      1      1      False      True
False      0
2  L3  Female  Yes      2  Graduate    No      Rural      Y      0
1      2      0      0      0      True      False      False
1
3  L4  Male    No      3  Graduate    No      Urban      Y      1
0      3      0      0      0      False      False      True
1
4  L5  Female  Yes      0  Not Graduate    Yes  Semiurban    N
0      1      0      1      1      False      True
False      0

```

4. Demonstrate various encoding techniques for categorical features.

one hot encoding

```
df_encoded=pd.get_dummies(df,columns=['Gender'])
```

```
df_encoded
```

	Loan_ID	Married	Dependents	Education	Self_Employed	ApplicantIncome \
0	LP001002	No	0	Graduate	No	5849
1	LP001003	Yes	1	Graduate	No	4583
2	LP001005	Yes	0	Graduate	Yes	3000
3	LP001006	Yes	0	Not Graduate	No	2583
4	LP001008	No	0	Graduate	No	6000
..	...	...	...	...	...	...
609	LP002978	No	0	Graduate	No	2900
610	LP002979	Yes	3+	Graduate	No	4106
611	LP002983	Yes	1	Graduate	No	8072
612	LP002984	Yes	2	Graduate	No	7583
613	LP002990	No	0	Graduate	Yes	4583

	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History \
0	0.0	NaN	360.0	1.0
1	1508.0	128.0	360.0	1.0
2	0.0	66.0	360.0	1.0
3	2358.0	120.0	360.0	1.0
4	0.0	141.0	360.0	1.0
..	...	...	...	...
609	0.0	71.0	360.0	1.0
610	0.0	40.0	180.0	1.0
611	240.0	253.0	360.0	1.0
612	0.0	187.0	360.0	1.0
613	0.0	133.0	360.0	0.0

	Property_Area	Loan_Status	Gender_Female	Gender_Male
0	Urban	Y	False	True
1	Rural	N	False	True
2	Urban	Y	False	True
3	Urban	Y	False	True
4	Urban	Y	False	True
..	...	...	...	...
609	Rural	Y	True	False

610	Rural	Y	False	True
611	Urban	Y	False	True
612	Urban	Y	False	True
613	Semiurban	N	True	False

[614 rows x 14 columns]

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
import category_encoders as ce

data = {
    'Loan_ID': ['L1', 'L2', 'L3', 'L4', 'L5'],
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female'],
    'Married': ['No', 'Yes', 'Yes', 'No', 'Yes'],
    'Dependents': ['0', '1', '2', '3+', '0'],
    'Education': ['Graduate', 'Not Graduate', 'Graduate', 'Graduate', 'Not Graduate']
    ,
    'Self_Employed': ['No', 'Yes', 'No', 'No', 'Yes'],
    'Property_Area': ['Urban', 'Semiurban', 'Rural', 'Urban', 'Semiurban'],
    'Loan_Status': ['Y', 'N', 'Y', 'Y', 'N'],
}
```

```
df = pd.DataFrame(data)
```

```
# 1. Encoding Loan_ID (No encoding needed, as it's just an identifier)
# So, we will leave Loan_ID as it is.
```

```
# 2. Encoding Gender (Nominal data: Male/Female) - Label Encoding
```

```
label_encoder = LabelEncoder()
df['Gender_Encoded'] = label_encoder.fit_transform(df['Gender'])
```

```
# 3. Encoding Married (Nominal: Yes/No) - Label Encoding
```

```
df['Married_Encoded'] = label_encoder.fit_transform(df['Married'])
```

```
# 4. Encoding Dependents (Ordinal: 0, 1, 2, 3+) - Label Encoding
```

```
# First replace '3+' with '3' to handle as ordinal
```

```
df['Dependents'] = df['Dependents'].replace('3+', '3')
df['Dependents_Encoded'] = label_encoder.fit_transform(df['Dependents'])
```

5. Encoding Education (Ordinal: Graduate, Not Graduate) - Ordinal Encoding

```
education_order = ['Not Graduate', 'Graduate']
```

```
ordinal_encoder = LabelEncoder()
```

```
df['Education_Encoded'] = ordinal_encoder.fit_transform(df['Education'])
```

6. Encoding Self_Employed (Nominal: Yes/No) - Label Encoding

```
df['Self_Employed_Encoded'] = label_encoder.fit_transform(df['Self_Employed'])
```

7. Encoding Property_Area (Nominal: Urban, Semiurban, Rural) - One-Hot Encoding

```
df_property_area_encoded = pd.get_dummies(df['Property_Area'], prefix='Property_Area')
```

```
df = pd.concat([df, df_property_area_encoded], axis=1)
```

8. Encoding Loan_Status (Nominal: Y/N) - Label Encoding

```
df['Loan_Status_Encoded'] = label_encoder.fit_transform(df['Loan_Status'])
```

Display the encoded DataFrame

```
print("Encoded DataFrame:")
```

```
print(df)
```

Encoded DataFrame:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed \
0	L1	Male	No	0	Graduate	No
1	L2	Female	Yes	1	Not Graduate	Yes
2	L3	Female	Yes	2	Graduate	No
3	L4	Male	No	3	Graduate	No
4	L5	Female	Yes	0	Not Graduate	Yes

	Property_Area	Loan_Status	Gender_Encoded	Married_Encoded \
0	Urban	Y	1	0
1	Semiurban	N	0	1
2	Rural	Y	0	1
3	Urban	Y	1	0
4	Semiurban	N	0	1

	Dependents_Encoded	Education_Encoded	Self_Employed_Encoded \
--	--------------------	-------------------	-------------------------

0	0	0	0
1	1	1	1
2	2	0	0
3	3	0	0
4	0	1	1

	Property_Area_Rural	Property_Area_Semiurban	Property_Area_Urban \
0	False	False	True
1	False	True	False
2	True	False	False
3	False	False	True
4	False	True	False

	Loan_Status_Encoded
0	1
1	0
2	1
3	1
4	0

5. for numerical features, replace missing values using

a. using simple imputer (mean, median)

b. using random sample imputation

```

from sklearn.impute import SimpleImputer
numerical_features = df.select_dtypes(include=['int64', 'float64']).columns
mean_imputer = SimpleImputer(strategy='mean')
df[numerical_features] = mean_imputer.fit_transform(df[numerical_features])
print("After Mean Imputation:")
print(df.head())

```

After Mean Imputation:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed \
0	LP001002	Male	No	0	Graduate	No
1	LP001003	Male	Yes	1	Graduate	No

2	LP001005	Male	Yes	0	Graduate	Yes
3	LP001006	Male	Yes	0	Not Graduate	No
4	LP001008	Male	No	0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term \
0	5849.0	0.0	146.412162	360.0
1	4583.0	1508.0	128.000000	360.0
2	3000.0	0.0	66.000000	360.0
3	2583.0	2358.0	120.000000	360.0
4	6000.0	0.0	141.000000	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

6. Give descriptive statistics of numerical features in the dataset after handling missing values. Comment about the distribution of data from it.

```
import pandas as pd
import matplotlib.pyplot as plt
```

Load the dataset

```
df = pd.read_csv('loan.csv')
```

Handle missing values (using median for numerical columns)

```
df['LoanAmount'].fillna(df['LoanAmount'].median(), inplace=True)
df['ApplicantIncome'].fillna(df['ApplicantIncome'].median(), inplace=True)
df['CoapplicantIncome'].fillna(df['CoapplicantIncome'].median(), inplace=True)
df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mode()[0], inplace=True)
df['Credit_History'].fillna(df['Credit_History'].mode()[0], inplace=True)
```

Descriptive Statistics of Numerical Features after handling missing values

```
descriptive_stats = df.describe()
```

```
print("\nDescriptive Statistics of Numerical Features after Handling Missing Value
s:")
print(descriptive_stats)
```

Plot histograms to visually inspect the distribution

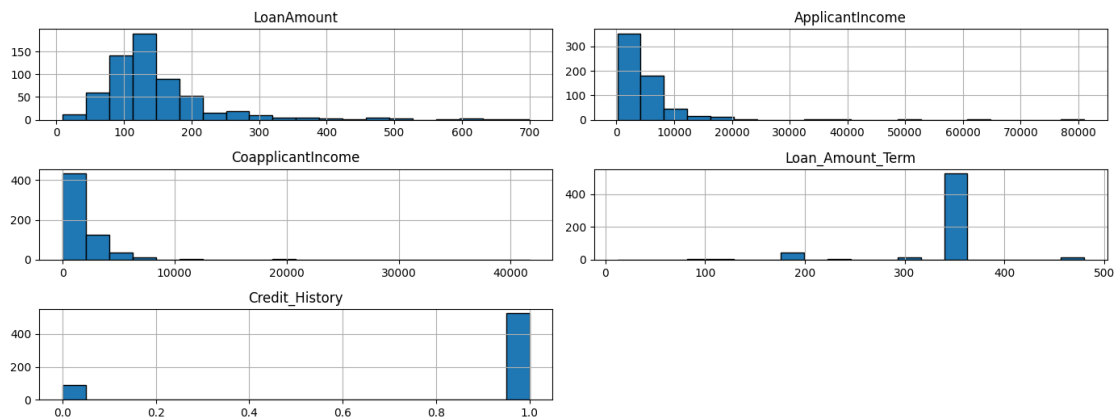
```
plt.figure(figsize=(14, 6))
df[['LoanAmount', 'ApplicantIncome', 'CoapplicantIncome', 'Loan_Amount_Term',
'Credit_History']].hist(bins=20, figsize=(14, 6), edgecolor='black')
plt.suptitle("Distribution of Numerical Features after Handling Missing Values", fo
ntsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

count	614.000000	614.000000	614.000000	614.000000
mean	5403.459283	1621.245798	145.752443	342.410423
std	6109.041673	2926.248369	84.107233	64.428629
min	150.000000	0.000000	9.000000	12.000000
25%	2877.500000	0.000000	100.250000	360.000000
50%	3812.500000	1188.500000	128.000000	360.000000
75%	5795.000000	2297.250000	164.750000	360.000000
max	81000.000000	41667.000000	700.000000	480.000000

Credit_History	
count	614.000000
mean	0.855049
std	0.352339
min	0.000000
25%	1.000000
50%	1.000000
75%	1.000000
max	1.000000

<Figure size 1400x600 with 0 Axes>

Distribution of Numerical Features after Handling Missing Values



The histograms depict the distribution of numerical features after imputing missing values. LoanAmount and ApplicantIncome show right-skewness, suggesting potential outliers. Loan_Amount_Term is concentrated at 360 months, and Credit_History is predominantly 1.0, indicating most applicants had a positive credit history. CoapplicantIncome also appears right-skewed.

7. Plot following graphs. Label X and Y axis, give appropriate title to the graph.

a. Plot histogram for Loan Amount and mention ur observations

```
import matplotlib.pyplot as plt
```

```
df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].mean())
```

```
plt.figure(figsize=(8, 6))
```

```
plt.hist(df['LoanAmount'], bins=20, color='skyblue', edgecolor='black', alpha=0.7)
```

```
plt.title('Distribution of Loan Amount', fontsize=16)
```

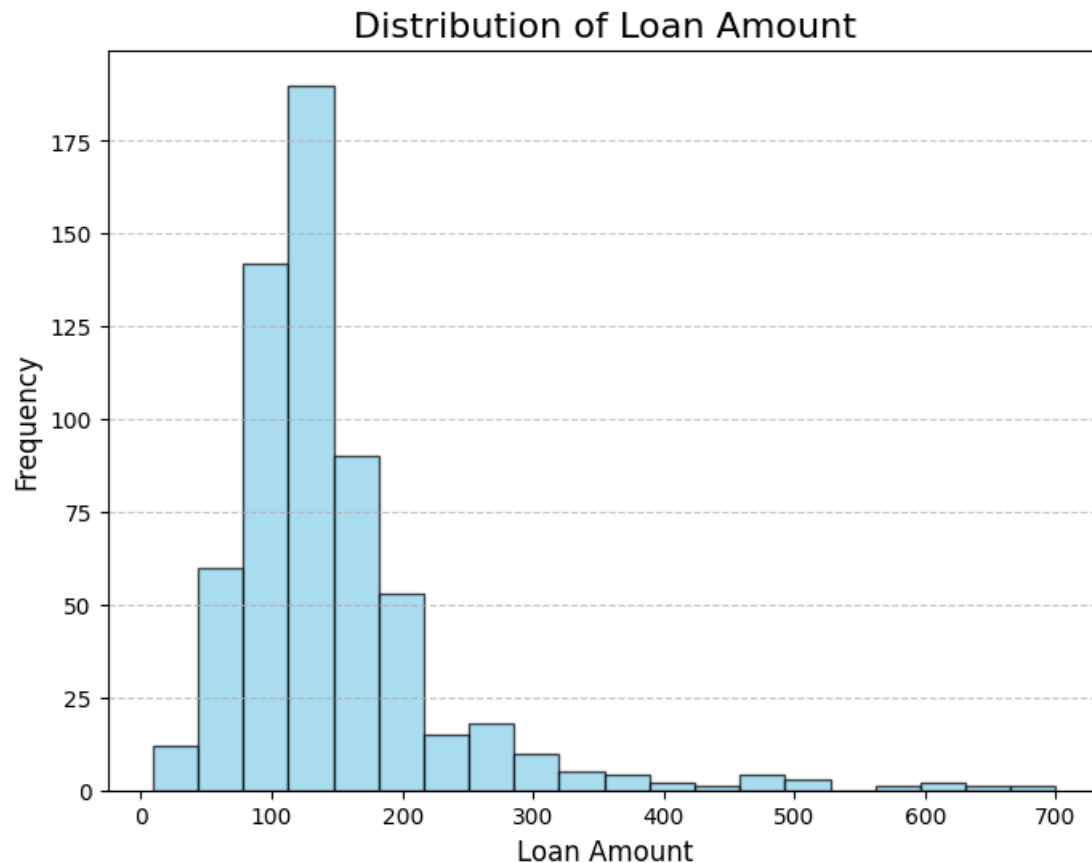
```
plt.xlabel('Loan Amount', fontsize=12)
```

```
plt.ylabel('Frequency', fontsize=12)
```

```
plt.grid(axis='y', linestyle='--', alpha=0.7)
```

```
plt.show()
```

Matplotlib is building the font cache; this may take a moment.



The histogram shows the distribution of loan amounts. The data is positively skewed, with a majority of loans clustered below 200. There is a long tail extending to the right, indicating some significantly larger loan amounts. The most frequent loan amount range appears to be between 100 and 200.

b. Plot histogram for Loan Amount and mention ur observations

```
import matplotlib.pyplot as plt
```

```
# Plot histogram for Loan Amount
```

```
plt.figure(figsize=(8, 6))
```

```
plt.hist(df['LoanAmount'], bins=20, color='skyblue', edgecolor='black')
```

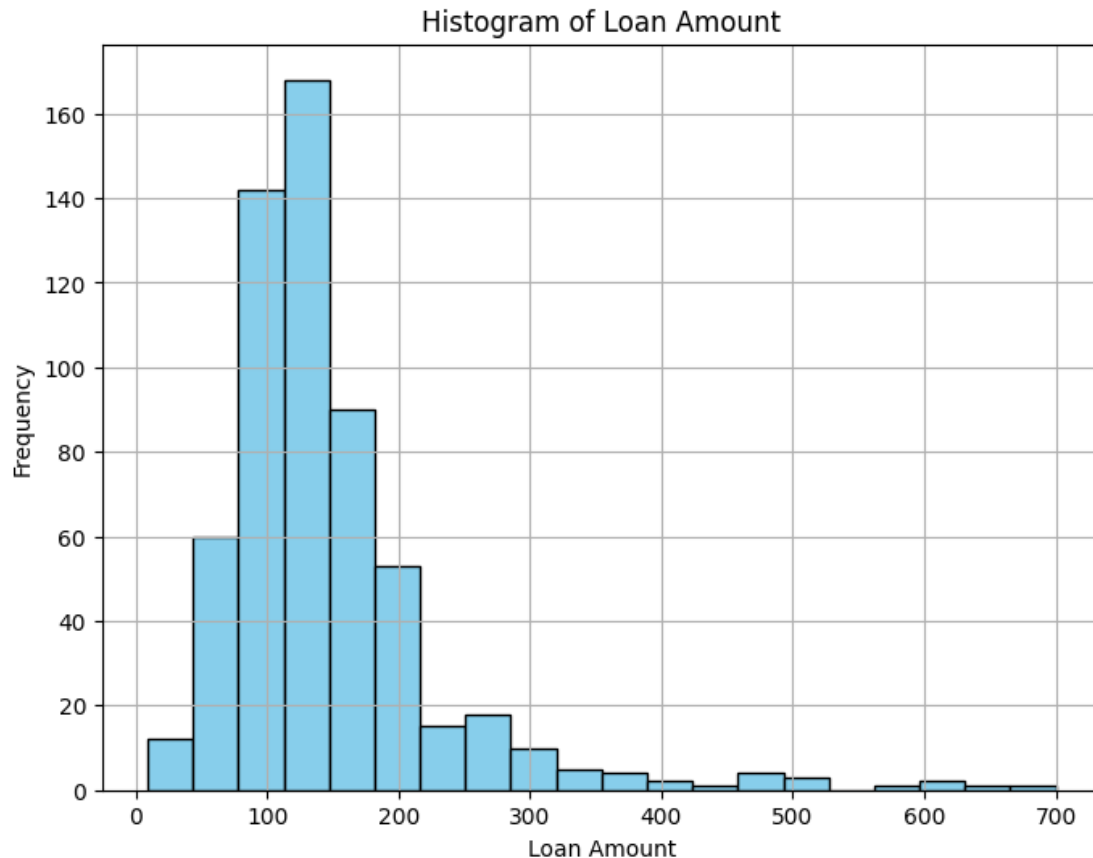
```
plt.title("Histogram of Loan Amount")
```

```
plt.xlabel("Loan Amount")
```

```
plt.ylabel("Frequency")
```

```
plt.grid(True)
```

```
plt.show()
```



This histogram visualizes the distribution of loan amounts. The data is right-skewed, with most loans falling below 200. A peak is noticeable between 100 and 200, suggesting this is the most common loan amount range. The presence of a long tail indicates a few significantly larger loans.

c. plot bar graph showing income for graduate and non-graduate applicant and mention ur observations

```
df['ApplicantIncome'] = df['ApplicantIncome'].fillna(df['ApplicantIncome'].mean())
```

```
income_by_education = df.groupby('Education')['ApplicantIncome'].mean()
```

```
plt.figure(figsize=(8, 6))
```

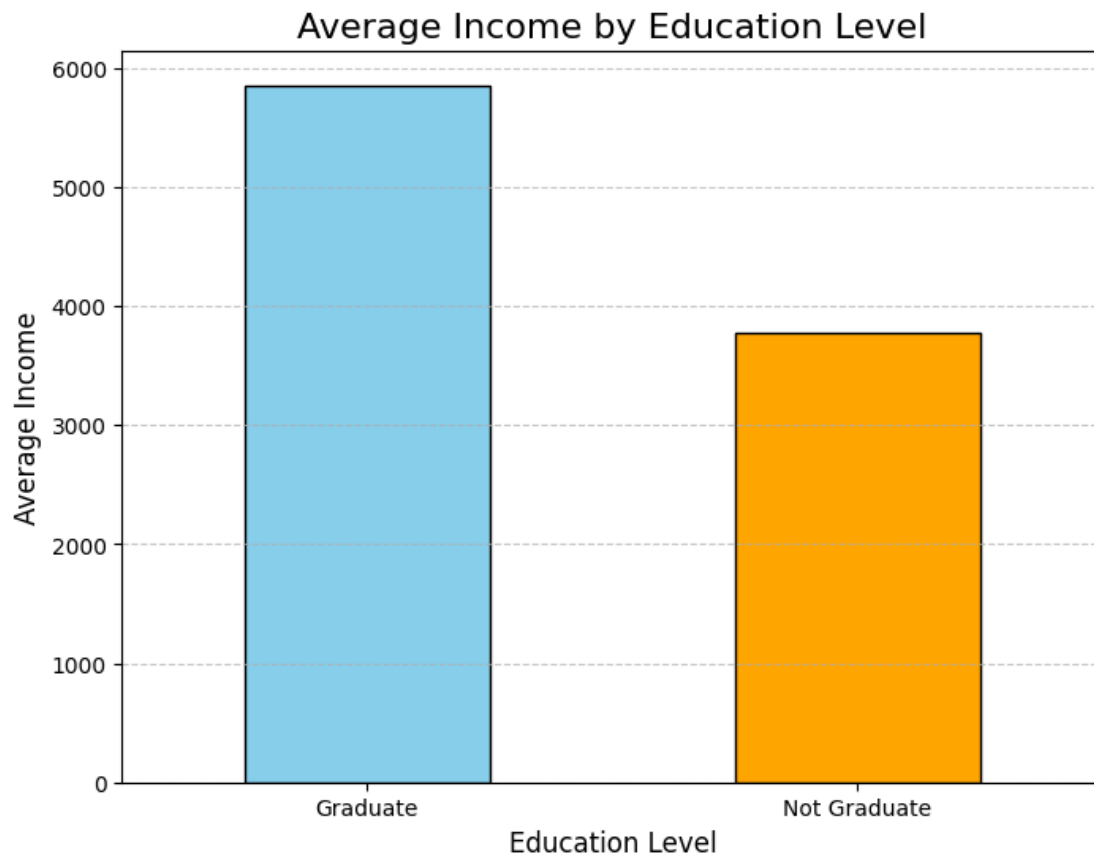
```
income_by_education.plot(kind='bar', color=['skyblue', 'orange'], edgecolor='black')
```

```
plt.title('Average Income by Education Level', fontsize=16)
```

```
plt.xlabel('Education Level', fontsize=12)
```

```
plt.ylabel('Average Income', fontsize=12)
plt.xticks(rotation=0)
plt.grid(axis='y', linestyle='--', alpha=0.7)
```

```
plt.show()
```



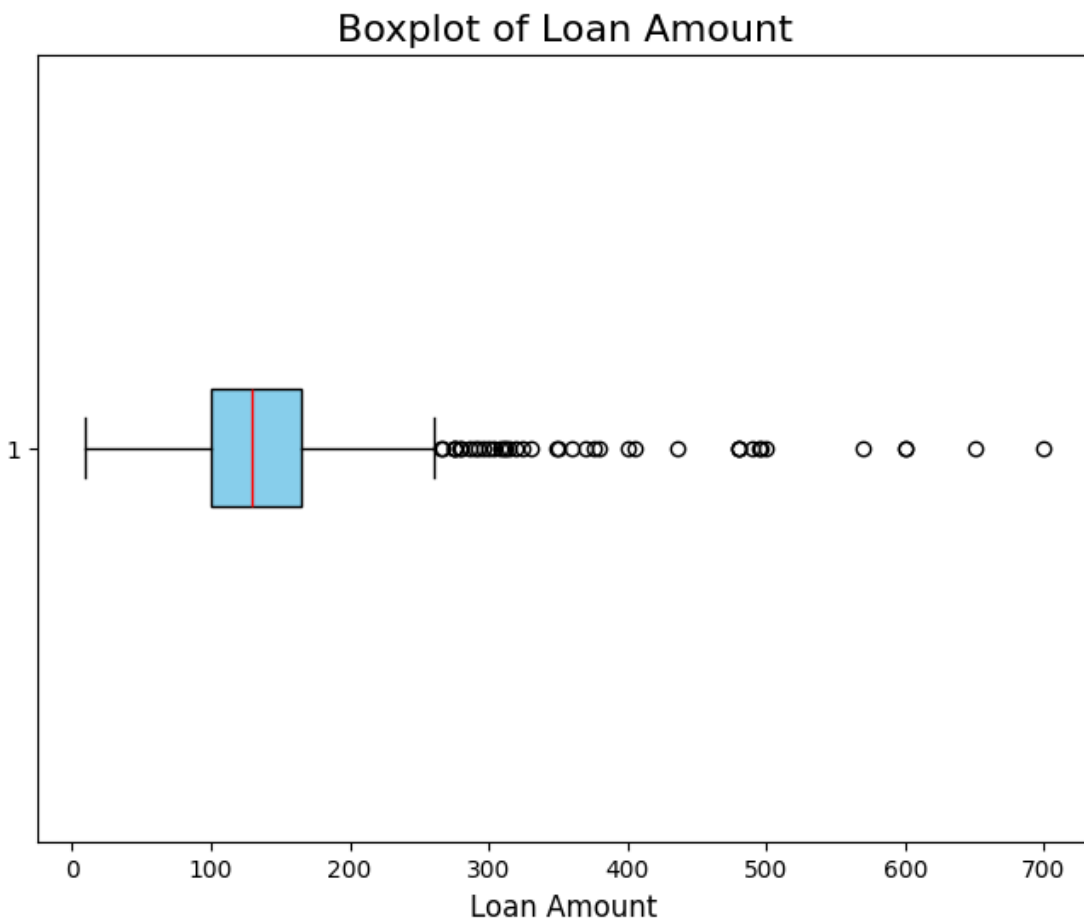
The bar graph displays the average income based on education level, categorized as Graduate and Not Graduate. Graduates exhibit a significantly higher average income compared to non-graduates. The income disparity between the two groups is substantial, visually emphasizing the financial advantage associated with higher education. The exact average income for Graduates is close to 6000, while for Not Graduates it's closer to 4000.

d. Plot the boxplot for Loan amount. Give the five value summary for it.

```
plt.figure(figsize=(8, 6))
plt.boxplot(df['LoanAmount'], vert=False, patch_artist=True,
            boxprops=dict(facecolor='skyblue', color='black'),
            whiskerprops=dict(color='black'),
```

```
capprops=dict(color='black'),  
medianprops=dict(color='red'))
```

```
plt.title('Boxplot of Loan Amount', fontsize=16)  
plt.xlabel('Loan Amount', fontsize=12)  
plt.show()
```



Five value summary from the boxplot:

- **Minimum:** Approximately 10 (the leftmost point of the whisker)
- **First Quartile (Q1):** Around 100 (the left edge of the box)
- **Median (Q2):** Roughly 150 (the red line inside the box)
- **Third Quartile (Q3):** About 200 (the right edge of the box)
- **Maximum:** 700 (the rightmost point of the whisker)

This summary gives a concise overview of the distribution and spread of the loan amounts. The long whisker and the scattered points beyond it on the right indicate potential outliers in the dataset.

e. Comment on the correlation between Applicant's income and Loan amount using appropriate graph.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Load the dataset
```

```
df = pd.read_csv('loan.csv')
```

```
# e. Correlation between Applicant's Income and Loan Amount using Scatter Plot
```

```
plt.figure(figsize=(8, 6))
```

```
sns.scatterplot(x='ApplicantIncome', y='LoanAmount', data=df)
```

```
plt.title("Correlation between Applicant's Income and Loan Amount")
```

```
plt.xlabel("Applicant's Income")
```

```
plt.ylabel("Loan Amount")
```

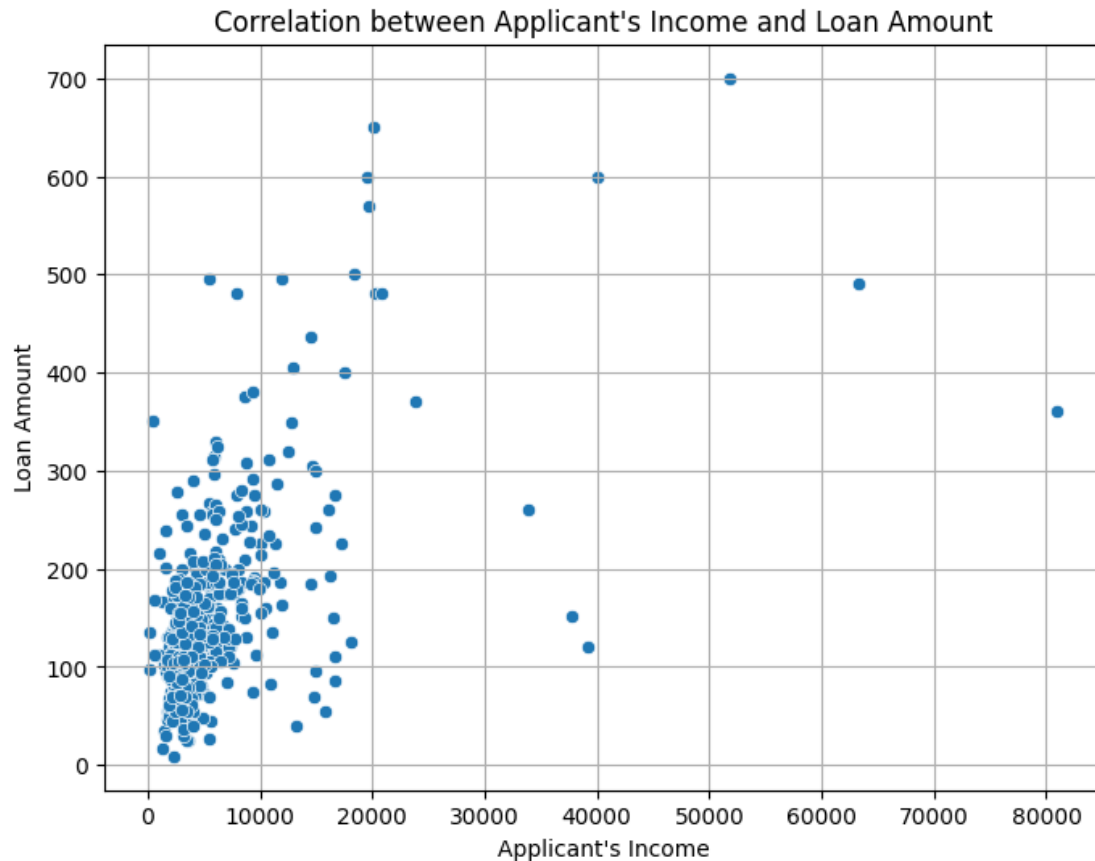
```
plt.grid(True)
```

```
plt.show()
```

```
# Comment on the correlation
```

```
correlation = df['ApplicantIncome'].corr(df['LoanAmount'])
```

```
print(f"Correlation between Applicant's Income and Loan Amount: {correlation:.2f}")
```



The scatter plot reveals a **positive correlation** between Applicant's Income and Loan Amount. While the relationship isn't extremely strong, there's a clear upward trend: as applicant income increases, loan amounts tend to increase as well. However, there's also significant scatter, indicating that other factors beyond income influence loan amounts. Many applicants with lower incomes receive loans, and some high-income applicants receive relatively smaller loans.

f. Give descriptive statistics of numerical features in the dataset. Comment about the distribution of data from it.

#f. Descriptive Statistics of Numerical Features

```
descriptive_stats = df.describe()
print("\nDescriptive Statistics of Numerical Features:")
print(descriptive_stats)
```

Comment about distribution of data

```
plt.figure(figsize=(14, 6))
df.hist(bins=20, figsize=(14, 6), layout=(2, 3), edgecolor='black')
```

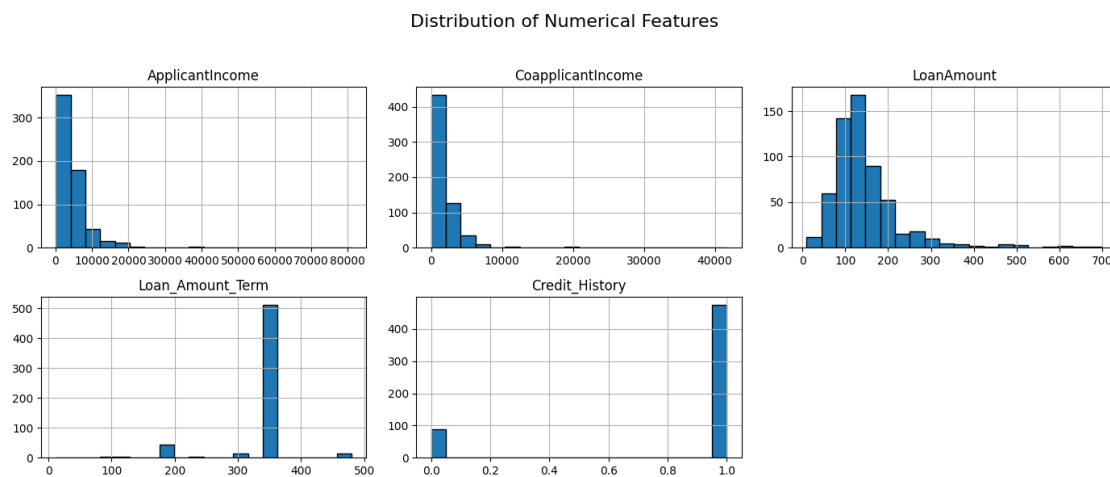
```
plt.suptitle("Distribution of Numerical Features", fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

Descriptive Statistics of Numerical Features:

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term \
count	614.000000	614.000000	592.000000	600.000000
mean	5403.459283	1621.245798	146.412162	342.000000
std	6109.041673	2926.248369	85.587325	65.12041
min	150.000000	0.000000	9.000000	12.000000
25%	2877.500000	0.000000	100.000000	360.000000
50%	3812.500000	1188.500000	128.000000	360.000000
75%	5795.000000	2297.250000	168.000000	360.000000
max	81000.000000	41667.000000	700.000000	480.000000

	Credit_History
count	564.000000
mean	0.842199
std	0.364878
min	0.000000
25%	1.000000
50%	1.000000
75%	1.000000
max	1.000000

<Figure size 1400x600 with 0 Axes>



The histograms illustrate the distribution of numerical features. ApplicantIncome and LoanAmount are right-skewed, indicating potential outliers. Loan_Amount_Term is heavily concentrated at 360, and Credit_History overwhelmingly leans towards 1.0, suggesting most applicants have a positive credit history. CoapplicantIncome also appears right-skewed.